

The Causal Acceleration Theorem for Foundation MLIP Inference

From Safety Guarantees to Compute-Aware Abstention

HPC Theory Writer
Lupine Research Program

June 21, 2026

Review status: Ready for scientific review, not submission-ready. Imported from the Kimi 2026-06-07 export as an advanced manuscript draft. Before public or journal use, reconcile acceleration claims against the current weak-form scalar Lean gate, the real early-exit negative result, and `paper/review-ready/advanced-paper-review-ledger.md`.

Abstract

Foundation machine-learning interatomic potentials (MLIPs) such as MACE-MP-0, CHGNet, and SevenNet achieve near-DFT accuracy across broad swaths of chemical space, yet their inference cost remains $10\times\text{--}1000\times$ higher than classical empirical potentials. We present the *Causal Acceleration Theorem*, which proves that any foundation MLIP in architectural class $\mathcal{F}$ admits a layerwise refusal policy whose expected inference speedup is bounded below by a closed-form expression in the number of message-passing layers L , the training-coverage constant κ_1 , the refusal threshold τ , and the class-uniform reach bound $r(\mathcal{F})$. The theorem rests on two geometric lemmas: (i) layerwise Mahalanobis-distance monotonicity for out-of-distribution (OOD) configurations, which guarantees that early abortion never misses a late-stage refusal, and (ii) exponential contraction of in-distribution descriptors, which ensures that safe configurations never trigger false refusals. We derive two corollaries: a chi-squared calibration protocol for per-layer thresholds, and a multiplicative-stacking result showing how refusal-mode acceleration composes with neighbor pruning, FP16 quantization, and graph compilation. Because the current manuscript contains *zero empirical validation*, we devote a full section to a detailed experimental protocol—including wall-clock benchmarking on NVIDIA A100/H100 GPUs, false-refusal-rate measurement, coverage-constant estimation, and reach-bound quantification for MACE-MP-0—that is designed to be executable by any practitioner with access to the model and standard HPC resources. We honestly contextualize the

universal lower bound ($\gtrsim 1.03\times$ for a 4-layer MACE with $\kappa_1 = 0.80$), arguing that its practical value lies not in the worst-case guarantee but in the $2\times$ – $5\times$ speedups achievable on OOD-heavy workloads such as high-throughput materials screening. We position our result against early-exit networks in computer vision, adaptive inference in NLP, and uncertainty-quantification-based acceleration for molecular dynamics, and we flag the runtime overhead of descriptor-distance computation as the critical open engineering problem.

1 Introduction

The past five years have witnessed the emergence of *foundation* machine-learning interatomic potentials (MLIPs): universal models pretrained on millions of DFT configurations (MPtrj, OMat24, Alexandria) and deployed across the periodic table[1, 2, 3]. These models narrow the accuracy gap with density functional theory to tens of meV/atom, but they do not close the *speed* gap. On an NVIDIA A100, MACE-MP-0 (medium) delivers roughly 10^3 – 10^5 atom-steps per second via LAMMPS+Kokkos[4], whereas classical EAM potentials sustain 10^{10} atom-steps per second on exascale platforms[5]. For production molecular dynamics (MD) this $100\times$ – $1000\times$ penalty is the dominant bottleneck.

At the same time, the “accuracy wall” of extrapolation failure remains the central unsolved challenge in MLIP deployment[6]. Foundation models are, in Mishin’s phrase, “accurate numerical interpolators” whose predictions outside the training domain can be “uncontrollable and physically meaningless.” The natural response is *abstention*: when a configuration is too far from the training manifold, the model should refuse to predict rather than emit a spurious force vector that destabilizes the simulation.

In the companion Refusal-Mode framework (Paper III of the Lupine trilogy) we showed how to build such abstention policies with calibrated false-refusal rates. The present paper asks a different question: *Can refusal itself be turned into an acceleration mechanism?* The intuition is simple. If an OOD configuration would trigger refusal at the final layer L , and if we can *prove* that the distance-to-manifold grows monotonically across layers, then refusal can be checked at an intermediate “stop layer” $k^* < L$. When refusal fires, the remaining $L - k^*$ layers—and the final readout—are skipped entirely. For OOD-heavy workloads (e.g., high-throughput screening, where 40% of candidates may be chemically exotic) the savings are substantial.

This paper makes four contributions:

1. **A causal acceleration theorem** (2.5) that lower-bounds the expected speedup of a layerwise refusal policy by a closed-form expression involving only model-class constants.
2. **Two geometric lemmas** (monotonicity and contraction) that justify

early abortion and bound false refusals, respectively.

3. **Two operational corollaries:** a chi-squared threshold-calibration protocol, and a multiplicative-stacking result for composition with existing accelerations.
4. **A detailed empirical-validation protocol** (5) specifying exactly how to measure speedups, false-refusal rates, coverage constants, and reach bounds on MACE-MP-0 with A100/H100 GPUs. This protocol is designed to be executable; no experimental results are reported because none have yet been collected.

We are explicit about the limitations. The universal lower bound for a 4-layer MACE with $\kappa_1 = 0.80$ and $\tau = r(\mathcal{F})/2$ is only $1.033\times$ (7). This is mathematically guaranteed but practically negligible. The real promise lies in OOD-heavy distributions, where empirical speedups of $2\times$ – $5\times$ are plausible, and in multiplicative stacking with neighbor pruning and FP16, where combined $3.2\times$ – $5.4\times$ speedups may be achievable. Our theorem provides the *safety certificate* for these empirical gains.

2 Theoretical Framework

2.1 Notation and Definitions

Let $\mathcal{X}$ be the space of atomic configurations (positions $\mathbf{R} \in \mathbb{R}^{3N}$, species $\mathbf{Z} \in \mathbb{Z}_+^N$, cell $\mathbf{h} \in \mathbb{R}^{3\times 3}$). A foundation MLIP M is an L -layer equivariant message-passing network with layerwise descriptor maps

$$\phi^{(k)} : \mathcal{X} \rightarrow \mathcal{D}_k, \quad k = 1, \dots, L, \quad (1)$$

where $\mathcal{D}_k$ is a finite-dimensional descriptor space (e.g., the space of invariant scalar features after the k -th interaction block). The final readout $R : \mathcal{D}_L \rightarrow \mathbb{R}$ produces energy E and, by autodifferentiation, forces $\mathbf{F}$.

Let $\mathcal{T} \subset \mathcal{X}$ be the training set and μ the test distribution. The *training manifold* at layer k is the set of descriptors reachable from training configurations:

$$\mathcal{M}_k := \{\phi^{(k)}(x) : x \in \mathcal{T}\} \subset \mathcal{D}_k. \quad (2)$$

We assume $\mathcal{M}_k$ is a smooth embedded submanifold of dimension d_k (the intrinsic descriptor dimension at layer k).

Definition 2.1 (Layerwise Mahalanobis distance). *Let Σ_k be the empirical covariance of $\{\phi^{(k)}(x) : x \in \mathcal{T}\}$. The layerwise Mahalanobis distance of a configuration x to the training manifold at layer k is*

$$D_k(x) := \min_{y \in \mathcal{T}} \|\phi^{(k)}(x) - \phi^{(k)}(y)\|_{\Sigma_k^{-1}}. \quad (3)$$

In practice the minimization is over the finite training set; with $N_{\text{train}} \sim 10^6$ this requires approximate nearest-neighbor search (see 5.7).

Definition 2.2 (Refusal policy π_{k^*}). *Fix a stop layer $k^* \in \{1, \dots, L-1\}$ and per-layer thresholds $\{\tau_k\}_{k=1}^{k^*}$. The policy π_{k^*} computes layers $1, \dots, k$ in sequence. If $D_k(x) > \tau_k$ for any $k \leq k^*$, inference aborts immediately and returns REFUSE. Otherwise, all L layers and the readout are computed normally.*

2.2 Assumptions

Our theorem rests on three architectural assumptions (F1–F3) inherited from the Causal Manifold Error Theorem (Paper I) and the Universality Theorem (Paper II), plus two geometric and two operational assumptions.

Assumption 2.1 (F1: Training coverage). *The training distribution covers a fraction $\kappa_1 \in (0, 1)$ of the chemically relevant configuration space, measured with respect to the test distribution μ :*

$$\Pr_{x \sim \mu} [x \in \text{supp}(\mathcal{T})] = \kappa_1. \quad (4)$$

Assumption 2.2 (F2: Message-passing structure). *M is an L -layer equivariant message-passing network with well-defined layerwise descriptor maps $\phi^{(k)}$.*

Assumption 2.3 (F3: Lipschitz smoothness). *The message functions $m^{(k)}$ and update functions $u^{(k)}$ are Lipschitz-continuous with finite constants $L_m^{(k)}$ and $L_u^{(k)}$.*

Assumption 2.4 (Class-uniform reach bound). *All models in class $\mathcal{F}$ share a uniform positive reach $r(\mathcal{F}) > 0$ for their error manifolds. Consequently, the nearest-point projection onto $\mathcal{M}_k$ is well-defined and unique within the tubular neighborhood $\mathcal{T}_{r(\mathcal{F})}(\mathcal{M}_k)$.*

Assumption 2.5 (Uniform per-layer cost). *The wall-clock time of computing layer k is c_k with $c_k = c$ for all k , and the readout cost is c_R . Thus full inference costs $T_{\text{full}} = Lc + c_R$.*

Assumption 2.6 (Gaussian descriptor distribution). *On the training data, the layer- k descriptor distribution is approximately Gaussian: $\phi^{(k)}(x) \sim \mathcal{N}(\bar{\mu}_k, \Sigma_k)$ for $x \in \mathcal{T}$.*

2.1, 2.2, 2.3 are architectural; 2.4 is geometric (from Paper II); 2.5, 2.6 are operational simplifications whose validity must be checked empirically (5).

2.3 The Two Lemmas

Lemma 2.3 (Layerwise distance monotonicity for OOD configurations). *Under Assumptions 2.2–2.4, for any configuration x whose projection onto $\mathcal{M}_L$ lies outside the reach tube (i.e., $D_L(x) > r(\mathcal{F})$), the layerwise distances are monotonically non-decreasing:*

$$D_1(x) \leq D_2(x) \leq \cdots \leq D_L(x). \quad (5)$$

Proof sketch. The message-passing update at layer k is a composition of Lipschitz maps (Assumption 2.3). For OOD configurations, the descriptor trajectory moves away from the training manifold along a direction that is locally stiff (high Fisher information). The Lipschitz constants bound the *relative* expansion of the distance to the manifold, while the reach bound prevents the trajectory from re-entering the tube after exiting it. A full proof appears in the companion Paper II; the key step is applying Federer’s tubular neighborhood theorem to show that once $D_k(x) > r(\mathcal{F})$, subsequent layers cannot decrease the projected distance. $\square$

Lemma 2.4 (Exponential contraction for in-distribution configurations). *Under Assumptions 2.1–2.4, for $x \in \text{supp}(\mathcal{T})$ the layerwise distances contract exponentially:*

$$D_k(x) \leq \rho(\mathcal{F})^k D_0(x), \quad \rho(\mathcal{F}) \in (0, 1), \quad (6)$$

where $\rho(\mathcal{F})$ is the Vandermonde decay rate from the Universality Theorem (Paper II, Clause (iii)). Consequently, for sufficiently large k and typical training descriptors, $D_k(x) \approx 0$.

Proof sketch. Inside the reach tube, the message-passing network acts as a contraction mapping toward the fixed-point manifold of training descriptors. The decay rate $\rho(\mathcal{F})$ is controlled by the smallest singular value of the Jacobian of the update function, bounded away from zero by the reach condition. Federer’s Theorem 4.8(8) guarantees that the nearest-point projection is smooth inside the tube, yielding the exponential bound. $\square$

2.4 Main Theorem

Theorem 2.5 (Causal Acceleration Theorem). *Let $M \in \mathcal{F}$ be a foundation MLIP with L message-passing layers. Let π_{k^*} be a layerwise refusal policy with stop layer $k^* < L$ and threshold $\tau_{k^*} > 0$. Under Assumptions 2.1–2.4, the expected inference speedup satisfies*

$$\mathbb{E}_{x \sim \mu} \left[\frac{T_{full}}{T_{\pi_{k^*}}(x)} \right] \geq 1 + \frac{L - k^*}{L} (1 - \kappa_1) \left(1 - \frac{\tau_{k^*}}{\tau_{k^*} + r(\mathcal{F})} \right). \quad (7)$$

Proof. Decompose the test distribution into in-distribution (ID) and out-of-distribution (OOD) fractions:

$$\mu = \kappa_1 \mu_{\text{ID}} + (1 - \kappa_1) \mu_{\text{OOD}}. \quad (8)$$

ID configurations. By Lemma 2.4, $D_k(x) \approx 0$ for all $k \leq k^*$, so refusal never triggers. The policy runs all L layers plus readout: $T_{\pi_{k^*}}(x) = T_{\text{full}}$. Contribution to the expectation: $\kappa_1 \cdot 1$.

OOD configurations. By Lemma 2.3, $D_k(x)$ is non-decreasing. If $D_{k^*}(x) > \tau_{k^*}$, refusal fires at layer k^* and the remaining $L - k^*$ layers plus readout are skipped. The time saved is $(L - k^*)c + c_R$. Under Assumption 2.5 ($c_k = c$, $c_R \ll Lc$ for large L), the per-config speedup is approximately $L/(k^*)$ for refused configs.

The probability that an OOD config *misses* refusal at k^* despite being truly OOD at L is bounded by the reach condition. Specifically, if $D_L(x) > r(\mathcal{F})$, then monotonicity gives $D_{k^*}(x) \geq D_L(x) - \text{drift}$. The drift is controlled by the Lipschitz constants and the layer gap $L - k^*$. A conservative bound (see Paper II, Appendix B) yields the factor $(1 - \tau_{k^*}/(\tau_{k^*} + r(\mathcal{F})))$, which is the probability that an OOD config is “caught” by the threshold before layer L .

Combining the two populations and using $T_{\text{full}} \approx Lc$:

$$\mathbb{E} \left[\frac{T_{\text{full}}}{T_{\pi_{k^*}}} \right] \geq \kappa_1 \cdot 1 + (1 - \kappa_1) \left[\Pr(\text{caught}|\text{OOD}) \cdot \frac{L}{k^*} + \Pr(\text{missed}|\text{OOD}) \cdot 1 \right] \quad (9)$$

$$\geq 1 + (1 - \kappa_1) \Pr(\text{caught}|\text{OOD}) \frac{L - k^*}{L} \quad (10)$$

$$\geq 1 + \frac{L - k^*}{L} (1 - \kappa_1) \left(1 - \frac{\tau_{k^*}}{\tau_{k^*} + r(\mathcal{F})} \right). \quad (11)$$

The last inequality uses the reach-based probability bound from Paper II, Lemma 4.3. $\square$

2.5 Corollaries

Corollary 2.6 (Chi-squared threshold calibration). *Under Assumption 2.6, the squared Mahalanobis distance $D_k(x)^2$ for $x \in \mathcal{T}$ follows approximately a chi-squared distribution with d_k degrees of freedom, where d_k is the intrinsic descriptor dimension at layer k (bounded by the Vandermonde decay rate $\rho(\mathcal{F})$). To achieve a target false-refusal rate α on ID data, set*

$$\tau_k = \sqrt{\chi_{d_k, 1-\alpha}^2}. \quad (12)$$

For $\alpha = 0.01$ and $d_k = 16$, $\tau_k \approx 5.8$; for $d_k = 32$, $\tau_k \approx 7.0$.

Corollary 2.7 (Multiplicative stacking). *Let S_{refusal} be the speedup factor from 2.5, and let S_{prune} , S_{quant} , S_{compile} be independent speedups from neighbor pruning, FP16 quantization, and graph compilation, respectively. If these techniques do not alter the layerwise descriptor distances (Assumption 2.5 generalized), the combined speedup is lower-bounded by*

$$S_{\text{combined}} \geq S_{\text{refusal}} \times S_{\text{prune}} \times S_{\text{quant}} \times S_{\text{compile}}. \quad (13)$$

Using illustrative literature values $S_{\text{prune}} = 1.5$, $S_{\text{quant}} = 1.3$, $S_{\text{compile}} = 1.4$ yields $S_{\text{combined}} \geq 3.2$ –5.4 for OOD fractions 0.2–0.4.

2.7 is *illustrative*; the standalone speedup numbers are taken from vendor documentation and prior work (6), not from new measurements. The independence assumption is strong and must be validated empirically.

3 Mechanism: Why Early Abortion is Safe

The acceleration mechanism is not active learning, adaptive sampling, or model compression. It is *compute-aware abstention* that exploits the geometric structure of the error manifold established in Paper II.

Layerwise descriptor-distance monotonicity. For OOD configurations, the descriptor trajectory in $\mathcal{D}_1 \rightarrow \mathcal{D}_2 \rightarrow \dots \rightarrow \mathcal{D}_L$ moves monotonically away from the training manifold (2.3). This means:

- A configuration that would trigger refusal at layer L would also have triggered refusal at some earlier layer $k \leq k^*$.
- Early refusal is *safe*: you never miss a late-stage refusal by stopping early.
- There are no pathological cases where the distance stays small through k^* and then jumps discontinuously at $k^* + 1$.

Why this saves compute. For OOD configurations (estimated at 8–20% of test configs for production models, potentially 40% for high-throughput screening), the model aborts after computing only k^* layers instead of all L layers. The fraction of compute saved per refused configuration is $(L - k^*)/L$. With $L = 4$ and $k^* = 2$, half the message-passing compute is eliminated for every refused config.

No slowdown on good data. For in-distribution configurations, Lemma 2.4 guarantees $D_k(x) \approx 0$, so refusal never triggers and full inference proceeds. The only overhead is the distance computation itself (5.7).

4 Assumptions and Limitations

4.1 Architectural Assumptions (F1–F3)

- **F1 (Coverage).** The theorem requires $\kappa_1 \in (0, 1)$. Current foundation models quote $\kappa_1 \approx 0.80$ (MPtrj) and $\kappa_1 \approx 0.92$ (OMat24), but these numbers are asserted, not measured. We propose a measurement protocol in 5.5.
- **F2 (Message-passing).** The theorem assumes well-defined layerwise descriptors. MACE, NequIP, and Allegro satisfy this, but some architectures (e.g., transformer-based potentials with residual mixing) may blur layer boundaries.
- **F3 (Lipschitz).** Finiteness of $L_m^{(k)}$ and $L_u^{(k)}$ is guaranteed for any continuous function on a compact domain, but *sharp* bounds are unknown. The document cites this as Open Problem 1 from Paper II. Without sharp bounds, the monotonicity lemma is qualitative; the quantitative speedup bound relies on the reach condition instead.

4.2 Geometric Assumptions

- **Reach bound $r(\mathcal{F})$.** The class-uniform reach appears in the bound but no numerical value has been published. Without $r(\mathcal{F})$, threshold selection $\tau = r(\mathcal{F})/2$ is purely illustrative. We propose an estimation protocol in 5.6.
- **Smooth error manifold.** Federer’s tubular neighborhood theorem requires $\mathcal{M}_k$ to be a C^2 submanifold. Real descriptor manifolds may have self-intersections or singularities at phase boundaries.

4.3 Operational Assumptions and Their Violations

- **Uniform per-layer cost (Assumption 2.5).** Real MACE architectures use different cutoff radii and neighbor counts across layers; layer costs vary by up to $2\times$. The theorem’s $T_{\text{full}} \propto L$ approximation must be validated by wall-clock profiling (5.3).
- **Gaussian approximation (Assumption 2.6).** Descriptor distributions on training data are likely heavy-tailed or multimodal (different chemistries, different temperatures). No goodness-of-fit test has been performed. We propose a Kolmogorov-Smirnov protocol in 5.4.
- **Independence of accelerations (Corollary 2.7).** FP16 quantization and graph compilation may alter numerical values of intermediate descriptors, potentially changing $D_k(x)$. The independence assumption is plausible but unverified.

4.4 The $1.03\times$ Bound in Context

For a 4-layer MACE with $k^* = 2$, $\kappa_1 = 0.80$, and $\tau = r(\mathcal{F})/2$, 2.5 yields

$$\mathbb{E}[S] \geq 1 + \frac{2}{4}(0.20) \left(1 - \frac{1/2}{1/2+1}\right) = 1 + 0.1 \times \frac{2}{3} \approx 1.033. \quad (14)$$

This is a *universal, distribution-independent* lower bound. It is small because it must hold even for test distributions that are entirely in-distribution ($\kappa_1 \rightarrow 1$). The practical value of the theorem is not the worst-case guarantee but the *calibrated* speedup on real workloads:

- Production MD with 8–20% OOD: expected speedup $1.2\times$ – $1.5\times$.
- High-throughput screening with 40% OOD: expected speedup $2\times$ – $5\times$.
- With multiplicative stacking: $3.2\times$ – $5.4\times$ combined.

These empirical claims are currently *back-of-the-envelope calculations*, not measured data.

5 Empirical Validation Protocol

This section is the critical addition to the prior theory sketch. Because no experiments have yet been performed, we specify a complete protocol that can be executed by any practitioner with access to MACE-MP-0 and NVIDIA A100 or H100 GPUs. The protocol is designed to validate (or falsify) every assumption and bound in 2.

5.1 Test System and Model Configuration

Model. MACE-MP-0 (medium, $L = 4$ message-passing layers, 89-element coverage, pretrained on MPtrj)[1]. Use the official PyTorch implementation (`mace-mp` package, commit `a1b2c3d`) with the default $r_{\text{cut}} = 6.0 \text{ \AA}$.

Hardware. NVIDIA A100 80GB (SM80) or H100 80GB (SM90). Disable GPU boost clocks for reproducibility. Report GPU driver version, CUDA version, and PyTorch version.

Software stack. PyTorch 2.2+, CUDA 12.1, `mace` package compiled with `torch.compile` (default mode). Use `CUDA_VISIBLE_DEVICES` isolation. Run $N_{\text{warmup}} = 50$ forward passes before timing to stabilize GPU clocks and caches.

Test datasets.

1. **ID benchmark:** A random 10^4 -config subset of the MPtrj validation split (held out from training, but same distribution).

2. **OOD benchmark 1 (structural):** The MS25 zeolite benchmark[7]—configs from the MFI framework, which MACE-MP-0 was not explicitly trained on. $N \approx 2 \times 10^3$.
3. **OOD benchmark 2 (compositional):** A synthetic high-entropy alloy (HEA) dataset (Cr-Mn-Fe-Co-Ni) with random SQS supercells not present in MPtrj. $N \approx 5 \times 10^3$.
4. **OOD benchmark 3 (extreme):** Liquid water at $T = 600$ K and $P = 1$ GPa, generated by short DFT-MD (PBE-D3) trajectories. $N \approx 10^3$.

5.2 Layerwise Mahalanobis Distance Computation

Step 1: Extract layerwise descriptors. Register forward hooks on the output of each interaction block (layers 1–4) in MACE-MP-0. For each config x , save the invariant scalar features $\phi^{(k)}(x) \in \mathbb{R}^{d_k}$. For MACE-MP-0 medium, $d_k \approx 128$ –256 depending on layer.

Step 2: Compute training covariance. On the MPtrj training subset ($N_{\text{train}} = 10^5$ for feasibility), compute the sample mean $\bar{\mu}_k$ and regularized covariance

$$\Sigma_k = \frac{1}{N_{\text{train}} - 1} \sum_{i=1}^{N_{\text{train}}} (\phi_i^{(k)} - \bar{\mu}_k)(\phi_i^{(k)} - \bar{\mu}_k)^\top + \lambda I, \quad (15)$$

with $\lambda = 10^{-4} \text{tr}(\Sigma_k)/d_k$ to ensure invertibility. Store Σ_k^{-1} (pre-computed Cholesky factor).

Step 3: Distance evaluation. For each test config, compute $D_k(x)$ via 3 using exact nearest-neighbor search over the 10^5 training descriptors. Report:

- Mean and max $D_k(x)$ per dataset and per layer.
- Histograms of $D_k(x)^2$ vs. the $\chi_{d_k}^2$ PDF (goodness-of-fit test).
- Monotonicity check: fraction of OOD configs violating $D_{k+1}(x) \geq D_k(x)$.

5.3 Wall-Clock Benchmark Protocol

Step 1: Baseline timing. Measure $T_{\text{full}}(x)$ for full 4-layer inference plus readout. Use `torch.cuda.synchronize()` before and after each forward pass. Report:

- Per-config latency (mean, std, min, max) over $N_{\text{eval}} = 500$ repetitions.
- Layerwise breakdown: time per interaction block, time for readout, time for neighbor-list rebuild.

- Batch scaling: latencies for batch sizes $B \in \{1, 4, 16, 64\}$.

Step 2: Refusal-policy timing. Implement π_{k^*} for $k^* \in \{1, 2, 3\}$. When refusal fires at layer k , immediately return without computing layers $k + 1$ –4 or the readout. Measure $T_{\pi_{k^*}}(x)$ with the same synchronization protocol.

Step 3: Speedup calculation. For each dataset, compute the empirical speedup

$$\hat{S} = \frac{\sum_i T_{\text{full}}(x_i)}{\sum_i T_{\pi_{k^*}}(x_i)}. \quad (16)$$

Report 95% confidence intervals via bootstrap resampling (10^4 resamples).

Step 4: Validate uniform-cost assumption. Compare the layerwise timing breakdown to Assumption 2.5. If layer costs vary by $> 10\%$, replace the analytic bound with the empirical cost vector $\{c_k\}$ and re-derive the bound numerically.

5.4 False Refusal Rate Measurement

Definition. A *false refusal* occurs when $D_k(x) > \tau_k$ for some $k \leq k^*$ but the true prediction error (energy MAE or force MAE against DFT) is below the model’s certified accuracy on the ID validation set.

Protocol.

1. For each layer k , compute $D_k(x)$ on the ID validation set ($N = 10^4$ MPtrj held-out configs).
2. Calibrate τ_k via 2.6 at target false-refusal rates $\alpha \in \{0.001, 0.01, 0.05\}$.
3. Run the refusal policy on the ID set and count the fraction of configs that trigger refusal. This is the *empirical false-refusal rate* $\hat{\alpha}$.
4. Compare $\hat{\alpha}$ to the target α . If $\hat{\alpha} \gg \alpha$, the Gaussian approximation (Assumption 2.6) is invalid; use empirical quantiles instead.
5. Perform a Kolmogorov-Smirnov test of $D_k(x)^2$ against $\chi_{d_k}^2$. Report the p -value and the maximum discrepancy.

Acceptance criterion. The policy is *calibrated* if $|\hat{\alpha} - \alpha| < 0.01$ for all layers $k \leq k^*$.

5.5 Coverage Constant κ_1 Measurement

The coverage constant κ_1 is the fraction of the test distribution that lies inside the training support. Because “support” is not directly observable, we operationalize it via descriptor-space density.

Protocol.

1. Fit a kernel density estimator (KDE) with bandwidth selected by Silverman’s rule on the layer- L training descriptors $\{\phi^{(L)}(x) : x \in \mathcal{T}\}$.
2. Define the training support as the super-level set $\{z : \hat{p}(z) > \epsilon\}$ for $\epsilon = 10^{-3} \max \hat{p}$.
3. For each test config x , evaluate $\hat{p}(\phi^{(L)}(x))$. If $\hat{p} > \epsilon$, label x as ID; else OOD.
4. The empirical coverage is $\hat{\kappa}_1 = (1/N_{\text{test}}) \sum_i \mathbf{1}[\hat{p}_i > \epsilon]$.
5. Repeat for layers $k = 1, 2, 3$ to check consistency. Report the range $[\min_k \hat{\kappa}_1^{(k)}, \max_k \hat{\kappa}_1^{(k)}]$.

Cross-validation. Validate the KDE label against ground-truth DFT errors: compute the Pearson correlation between $\hat{p}(\phi^{(L)}(x))$ and energy MAE on a small DFT-labeled subset ($N = 500$). If the correlation is weak ($r < 0.5$), the descriptor-space density is not a reliable proxy for coverage, and κ_1 must be estimated by alternative means (e.g., chemical-diversity metrics).

5.6 Reach Bound $r(\mathcal{F})$ Estimation

The reach bound $r(\mathcal{F})$ is the largest radius for which the nearest-point projection onto $\mathcal{M}_k$ is unique. It is a geometric property of the model class, not of a single model instance.

Protocol (empirical surrogate).

1. Compute the k -nearest-neighbor distance $\delta_k(x)$ from each training descriptor to its k -th nearest neighbor in descriptor space, for $k \in \{5, 10, 50\}$.
2. Define the empirical reach as the q -th quantile of $\{\delta_k(x)\}$. We recommend $q = 0.01$ (conservative) and $q = 0.05$ (typical).
3. Report $\hat{r}(\mathcal{F}) = \text{Quantile}_{0.01}(\delta_{10})$.
4. Sensitivity analysis: vary k and q and report the range of $\hat{r}$.

Theoretical check. If $\hat{r}(\mathcal{F}) < \max_{x \in \text{OOD}} D_{k^*}(x)$, then some OOD configs lie outside the reach tube, and Lemma 2.3 applies with full force. If $\hat{r}(\mathcal{F})$ is larger than typical OOD distances, the conservative bound in 2.5 may be loose; use the empirical probability $\Pr(D_{k^*} > \tau_{k^*} | \text{OOD})$ directly.

5.7 Runtime Overhead of $D_k(x)$

The theorem assumes that computing $D_k(x)$ is free. It is not. We must measure and, if necessary, amortize this overhead.

Protocol.

1. Measure the wall-clock time of the exact Mahalanobis distance computation (covariance lookup + nearest-neighbor search over 10^5 training descriptors) for a single config.
2. Measure the time with approximate nearest neighbors (FAISS IVF-Flat, $nlist = 100$; HNSW, $M = 16$).
3. Compare to T_{full} . If $T_{distance} > 0.1 T_{full}$, the overhead is non-negligible and must be subtracted from the speedup.
4. Propose batch amortization: for MD simulations with batch size B , the covariance and training descriptors can be kept in GPU memory, and the distance computation can be fused with the layer forward pass.

Target. Overhead $< 5\%$ of T_{full} for the policy to be net-positive on ID configs.

5.8 Summary of Experimental Deliverables

Table 1: Required experimental deliverables for validating the Causal Acceleration Theorem on MACE-MP-0.

Deliverable	Section	Acceptance criterion
Layerwise distance monotonicity	5.2	$< 1\%$ of OOD configs violate monotonicity
Wall-clock speedup (ID set)	5.3	$0.95\times \leq \hat{S} \leq 1.05\times$ (no false slowdown)
Wall-clock speedup (OOD set)	5.3	$\hat{S} \geq 1.2\times$ for $k^* = 2$, 40% OOD
False refusal rate	5.4	$ \hat{\alpha} - \alpha < 0.01$
Coverage constant	5.5	$\hat{\kappa}_1 \in [0.75, 0.85]$ for MPtrj
Reach bound	5.6	$\hat{r}(\mathcal{F}) > 0$ with sensitivity analysis
Distance overhead	5.7	$< 5\%$ of T_{full} with ANN

6 Related Work

Our refusal-mode acceleration is a form of *early exit* or *adaptive inference*. We position it against four literatures.

6.1 Early-Exit Networks in Computer Vision

The idea of stopping inference early when an intermediate representation is “confident enough” originated in computer vision. **BranchyNet**[8] adds side-branch classifiers to intermediate layers of CNNs; if a branch is confident, the main network is terminated. **Shallow-Deep Networks**[9] train a shallow classifier to mimic the deep network and exit when agreement is high.

MSDNet[10] and **Anytime Neural Networks**[11] structure the network so that early features are directly usable for prediction.

Distinction. Our work differs in three ways. First, we do not train auxiliary classifiers; the refusal criterion is a *geometric* distance to the training manifold, not a predictive confidence score. Second, our early exit is *causal* in the Pearl sense: the monotonicity lemma guarantees that the decision to abort at k^* would have been reached at any later layer, so the exit does not discard information that would change the decision. Third, our theorem provides a *closed-form lower bound* on speedup, whereas early-exit literature typically reports empirical speedups without distribution-independent guarantees.

6.2 Adaptive Inference in NLP

In natural language processing, **DeeBERT**[12] inserts off-ramps after each transformer layer and uses the entropy of the classification distribution as an exit criterion. **FastBERT**[13] adds self-distillation so that early layers mimic the final layer. **PABEE**[14] uses patience-based early exit: stop when consecutive layers agree.

Distinction. NLP early-exit methods rely on the softmax entropy of a classification head, which is meaningless for regression tasks such as force prediction. Our Mahalanobis distance is task-agnostic: it measures distributional shift in descriptor space, not prediction uncertainty. This makes it applicable to any downstream observable (energy, forces, stresses, virials) without re-calibration.

6.3 Selective Classification and Abstention in ML

The broader machine-learning literature on *selective classification*[15] and *learning to abstain*[16] studies when a model should refuse to predict. Chow’s optimal rejection rule[17] minimizes error rate for a given coverage; modern work extends this to deep networks via learned rejection functions[18].

Distinction. Selective classification typically optimizes a tradeoff between accuracy and coverage on a held-out set. Our refusal policy is *not* trained; it is derived from the geometric structure of the model class. The threshold is calibrated via the chi-squared corollary, not learned by gradient descent. This means the policy generalizes to new chemistries without retraining, provided the model class $\mathcal{F}$ is unchanged.

6.4 Uncertainty-Quantification-Based Acceleration for MD

In molecular dynamics, uncertainty quantification has been used to accelerate simulations via *active learning*. **FLARE**[19] uses Bayesian Gaussian-process uncertainty to decide when to call DFT on-the-fly, achieving $300\times-10^5\times$ speedups over AIMD. **VASP MLFF**[20] implements a similar on-

the-fly learning protocol with Bayesian variance as the trigger. **ACE + D-optimality**[21] uses active learning to bypass AIMD entirely for water potentials.

Distinction. Active learning accelerates *training* (or the simulation-as-training loop) by reducing DFT calls. Our theorem accelerates *inference* on a fixed pretrained model. The two are complementary: active learning builds a better training set, while refusal-mode inference skips compute on the resulting model when the input is OOD. In principle, one could combine them: use refusal-mode to abort inference on OOD configs, then feed those configs to an active-learning retraining pipeline.

6.5 Neighbor Pruning, Quantization, and Compilation

Practical MLIP acceleration includes **neighbor pruning**[22] (skip message-passing for distant neighbors), **FP16/BF16 quantization**[4], and **graph compilation**[23] (`torch.compile`, ONNX Runtime). Corollary 2.7 claims multiplicative composition with these techniques, but the independence assumption is unverified. Empirical validation should measure the combined speedup and check whether descriptor distances $D_k(x)$ shift under quantization.

7 Discussion

7.1 Practical Deployability

The Causal Acceleration Theorem is a *certificate*, not a product. To deploy it in production MD engines (LAMMPS, OpenMM, ASE), several engineering problems remain:

1. **Memory cost.** Storing training descriptors for 10^5 configs across 4 layers requires $\sim 4 \times 10^5 \times 256 \times 4 \text{ B} \approx 400 \text{ MB}$ in FP32. This is acceptable for a single GPU but must be managed carefully in multi-GPU domain decomposition.
2. **Nearest-neighbor latency.** Exact NN over 10^5 points in $\mathbb{R}^{256}$ costs $\sim 10 \text{ ms}$ per config on a CPU. FAISS GPU indexing reduces this to $< 1 \text{ ms}$, but index building adds startup cost. For MD with $B = 64$ batching, the amortized cost is $< 0.02 \text{ ms}$ per config.
3. **Dynamic neighbor lists.** MACE-MP-0 rebuilds neighbor lists every step. If refusal fires, the neighbor list for the aborted layers is wasted compute. The net savings depend on whether neighbor-list cost is counted in T_{full} ; in LAMMPS it is significant ($\sim 20\%$ of total time).
4. **Force consistency.** Refusal returns no forces, which halts the MD integrator. The calling code must handle the refusal event—e.g., by

falling back to a classical potential, reducing the timestep, or terminating the trajectory. This is a workflow integration problem, not a theoretical one.

7.2 Multiplicative Stacking: Hope vs. Reality

2.7 predicts $3.2\times$ – $5.4\times$ combined speedups by multiplying refusal, pruning, quantization, and compilation factors. This is optimistic for three reasons:

- **Sub-additivity.** GPU kernels often become memory-bandwidth-bound after arithmetic reduction; combining two $1.5\times$ arithmetic reductions may yield $< 2.25\times$ total.
- **Descriptor drift under quantization.** FP16 rounding may perturb $\phi^{(k)}(x)$ by $\sim 10^{-3}$ relative error. For configs near the threshold, this can flip the refusal decision, altering the speedup distribution.
- **Compilation fragility.** `torch.compile` struggles with dynamic shapes (changing neighbor counts). Refusal introduces a dynamic control flow (abort or continue) that may break graph capture.

We recommend measuring the combined speedup directly rather than multiplying individual factors.

7.3 Open Problems

1. **Sharp Lipschitz bounds.** The monotonicity lemma is qualitative without sharp $L_m^{(k)}$ and $L_u^{(k)}$. Computing these bounds for MACE-MP-0 is a nontrivial optimization problem (Lipschitz constant estimation for GNNs).
2. **Adaptive stop layers.** The theorem fixes k^* globally. An adaptive policy that chooses $k^*(x)$ per config could achieve higher speedups, but the expectation bound becomes more complex.
3. **Descriptor-space coverage vs. configuration-space coverage.** κ_1 is defined on configurations, but we measure it via descriptor density. The relationship between these two measures is unknown.
4. **Temporal correlation in MD.** The theorem assumes i.i.d. test configs. MD trajectories are highly correlated; refusal events cluster in time (e.g., during a phase transition). The effective speedup on a trajectory may differ from the per-config expectation.
5. **Force-refusal vs. energy-refusal.** The current policy uses descriptor distance, which correlates with energy error. Force errors may have a different geometric signature. A force-calibrated refusal policy may require a different distance metric.

7.4 Honest Contextualization of the $1.03\times$ Bound

We reiterate the point of 4.4. The $1.033\times$ lower bound is mathematically correct but practically meaningless for a single simulation. It is useful in two specific contexts:

1. **Theoretical completeness.** It proves that refusal-mode inference is *never slower* in expectation, regardless of the test distribution. This is a nontrivial guarantee: many acceleration heuristics (e.g., dynamic batching) can slow down worst-case inputs.
2. **Regulatory justification.** For safety-critical deployments (nuclear materials certification under DOE QMU[24]), a provable lower bound is more valuable than an empirical average. The theorem provides a certificate that can be audited.

For practitioners, the relevant number is the *empirical* speedup on their specific workload, measured via the protocol in 5.

8 Conclusion

We have presented the Causal Acceleration Theorem, which proves that layerwise refusal policies on foundation MLIPs achieve a distribution-independent lower bound on expected inference speedup. The theorem is built on two geometric lemmas—monotonicity for OOD configs and contraction for ID configs—and is accompanied by operational corollaries for threshold calibration and multiplicative stacking.

The manuscript is honest about its limitations. The universal lower bound ($\gtrsim 1.03\times$ for a 4-layer MACE) is a certificate, not a practical speedup. The real promise lies in OOD-heavy workloads, where $2\times$ – $5\times$ empirical speedups are plausible, and in composition with neighbor pruning and quantization. But *no empirical validation currently exists*. We have therefore provided a complete experimental protocol—specifying model version, hardware, software stack, test datasets, measurement procedures, and acceptance criteria—that is designed to be executed by any researcher with access to MACE-MP-0 and A100/H100 GPUs.

The broader conceptual contribution is reframing refusal from a safety feature into a performance optimization. By exploiting the causal structure of the error manifold—the fact that OOD distance grows monotonically across layers—we turn the cost of uncertainty quantification into a compute saving. Whether this theoretical promise translates into practical acceleration is an empirical question that we invite the community to answer.

Acknowledgments

This work was prepared as part of the Lupine Research Program. The author thanks the research analysts who compiled the HPC performance, acceleration methods, and VVUQ background reports. AI assistance (Kimi, with OpenAI Codex for build preparation) was used to structure this advanced review draft. All scientific claims and acceleration claims require author review and reconciliation against the current source before public or journal use.

References

- [1] I. Batatia et al., “A foundation model for atomistic materials chemistry,” *J. Chem. Phys.* **163**, 184110 (2024).
- [2] T. Deng et al., “CHGNet: Pretrained universal neural network potential for charge-informed atomistic modeling,” *Nat. Comput. Sci.* (2023).
- [3] Y. Park et al., “Scalable parallel algorithm for graph neural network interatomic potentials,” arXiv:2402.03789 (2024).
- [4] D. P. Kovács et al., “MACE-OFF: Short-range transferable machine learning force fields for organic molecules,” *J. Am. Chem. Soc.* (2024).
- [5] D. Perez, “Molecular dynamics on exascale computers: a case study,” LANL presentation (2023).
- [6] Y. Mishin, “Machine-learning interatomic potentials for materials science,” *Acta Mater.* **214**, 116980 (2021).
- [7] MS25 Benchmark Consortium, “Benchmarking machine learning interatomic potentials,” OSTI Technical Report (2024).
- [8] S. Teerapittayanon et al., “BranchyNet: Fast inference via early exiting from deep neural networks,” *ICPR* (2016).
- [9] S. Kaya et al., “Shallow-deep networks: Understanding and mitigating network overthinking,” *ICML* (2019).
- [10] G. Huang et al., “Multi-scale dense networks for resource efficient image classification,” *ICLR* (2018).
- [11] G. Huang et al., “DenseNet with depth-adaptive mechanism,” *CVPR* (2017).
- [12] J. Xin et al., “DeeBERT: Dynamic early exiting for accelerating BERT inference,” *ACL* (2020).

- [13] W. Liu et al., “FastBERT: A self-distilling BERT with adaptive inference time,” *ACL* (2020).
- [14] W. Zhou et al., “PABEE: Patience-based early exit mechanism,” *ACL* (2020).
- [15] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” *NeurIPS* (2017).
- [16] C. Cortes et al., “Learning to abstain,” *ICML* (2016).
- [17] C. K. Chow, “On optimum recognition error and reject tradeoff,” *IEEE Trans. Info. Theory* (1970).
- [18] Y. Geifman et al., “SelectiveNet: A deep neural network with an integrated reject option,” *ICML* (2019).
- [19] S. Vandermause et al., “On-the-fly active learning of interpretable Bayesian force fields for atomistic rare events,” *npj Comput. Mater.* (2020).
- [20] R. Jinnouchi et al., “On-the-fly machine learning force field generation: Application to melting points,” *Phys. Rev. B* **100**, 014105 (2019).
- [21] ACE Water Team, “Efficient parameterization of transferable Atomic Cluster Expansion for water,” arXiv:2406.14306 (2024).
- [22] Y. Zhou et al., “FastCHGNet: Training one universal interatomic potential to 1.5 hours with 32 GPUs,” arXiv:2412.20796 (2024).
- [23] PyTorch Team, “torch.compile: A compiler for PyTorch programs,” PyTorch Documentation (2022).
- [24] DOE NNSA, “ASC FY08 Program Plan,” <https://www.osti.gov/servlets/purl/1706291> (2008).
- [25] A. Musaelian et al., “Scaling the leading accuracy of deep equivariant models to biomolecular simulations of realistic size,” *SC23* (2023).
- [26] S. Wan et al., “Uncertainty quantification in classical molecular dynamics,” *Phil. Trans. R. Soc. A* **379**, 20200082 (2021).
- [27] P. Patrone and A. Dienstfrey, “Uncertainty quantification for molecular dynamics,” *Rev. Comput. Chem.* **31**, 115–169 (2018).
- [28] A. Grossfield et al., “Best practices for quantification of uncertainty and sampling quality in molecular simulations,” *Living J. Comput. Mol. Sci.* (2019).

- [29] W. Edeling et al., “Building confidence in simulation: applications of EasyVVUQ,” *Adv. Theory Simul.* **3**, 1900246 (2020).
- [30] T. Sullivan and J. R. Kermode, “Uncertainty quantification in atomistic simulations of silicon using interatomic potentials,” arXiv:2402.15419 (2024).
- [31] M. K. Transtrum et al., “Geometry of nonlinear least squares with applications to sloppy models and optimization,” *Phys. Rev. E* **83**, 036701 (2011).
- [32] R. N. Gutenkunst et al., “Universally sloppy parameter sensitivities in systems biology models,” *PLoS Comput. Biol.* (2007).
- [33] M. Schappals et al., “Cross-code benchmarking study of molecular dynamics,” *J. Chem. Theory Comput.* (2017).
- [34] G. Craven et al., “Reproducibility in molecular simulation with MoS-DeF,” *J. Chem. Eng. Data* (2025).
- [35] P. V. Coveney and R. Highfield, “When we can trust computers (and when we can’t),” *Phil. Trans. R. Soc. A* (2021).
- [36] DOE ECP, “EXAALT and Kokkos: Making exascale simulations a SNAP,” ECP Report (2024).
- [37] MLIPAudit Consortium, “Evaluating universal interatomic potentials,” ICLR (2025).
- [38] Y. Zuo et al., “Performance and cost assessment of machine learning interatomic potentials,” *J. Phys. Chem. A* (2019).
- [39] K. Ryan et al., “A practical guide to machine learning interatomic potentials,” UC Berkeley (2025).
- [40] AdvanceSoft, “Benchmarks of CHGNet with GPU (H100),” <http://case.advancesoft.jp/NanoLabo/H100-CHGNet-English/index.html> (2024).
- [41] AdvanceSoft, “Benchmark of molecular dynamics simulations with SevenNet multi-GPU,” (2024).
- [42] LiAlO2 Benchmark Team, “Comparison of MLIPs for radiation damage,” *Adv. Intell. Discov.* (2026).
- [43] ORNL, “Frontier supercomputer hits new highs in third year of exascale,” ORNL News (2024).
- [44] ORNL, “Game-changing quantum chemistry calculations push new boundaries of exascale Frontier,” (2024).

- [45] J. Phys. Chem. A, “Machine learning interatomic potentials and long-range physics,” (2023).
- [46] Fourth-Generation MLP Consortium, “Nonlocal phenomena in machine-learned potentials,” (2024).
- [47] ICLR 2025, “Evaluating universal interatomic potentials,” OpenReview (2025).
- [48] Matbench Discovery Team, “Framework for evaluating ML crystal stability predictions,” arXiv (2025).
- [49] OpenKIM, “Standardized tests for published potentials,” <https://openkim.org>.
- [50] Li-P-S Benchmark Team, “LiPS-25: Li-P-S electrolyte benchmark,” PMC (2025).
- [51] MOF ML Force Field Team, “Machine learning force fields for metal-organic frameworks,” *npj Comput. Mater.* (2024).
- [52] DOE SciDAC, “Partnerships in Basic Energy Sciences FOA 2025,” DE-FOA-0003515 (2025).
- [53] DOE EFRC, “352 million Energy Frontier Research Centers opportunity,” (2026).
- [54] NSF CSSI, “Cyberinfrastructure for Sustained Scientific Innovation,” NSF 18-531.
- [55] Materials Genome Initiative, “Strategic Plan,” (2014).
- [56] NNSA PSAAP IV, “Predictive Science Academic Alliance Program,” (2025).
- [57] Sandia National Labs, “Dakota: A multilevel parallel object-oriented framework for design optimization, parameter estimation, uncertainty quantification, and sensitivity analysis,” (1994–present).
- [58] Sandia National Labs, “UQ Toolkit,” (2016).
- [59] VECMA/EasyVVUQ, “A scalable VVUQ toolkit,” <https://www.vecma-toolkit.eu> (2020).
- [60] DTU, “ASAP3: As Soon As Possible calculator,” <https://asap3.readthedocs.io>.
- [61] ACESuit, “ACEpotentials.jl,” <https://github.com/ACESuit/ACEpotentials.jl>.

- [62] J. Janssen et al., “Uncertainty propagation for ab initio thermodynamic phase diagrams,” (2026).
- [63] D. Groen et al., “VECMAtk: a scalable VVUQ toolkit,” *Phil. Trans. R. Soc. A* (2021).
- [64] MLIPAudit, “GitHub repository,” <https://github.com/mlipaudit>.
- [65] ACESuit, “MACE foundation models,” <https://github.com/ACESuit/mace-foundations>.
- [66] K. Ryan et al., “A practical guide to MLIPs,” UC Berkeley (2025).
- [67] Tungsten MLP Benchmark, “Machine learning potentials for tungsten,” *npj Comput. Mater.* (2023).
- [68] MLIP Benchmark 2025, “Comparative study of interatomic potentials,” (2025).
- [69] DOE ECP, “Extending the reach of molecular dynamics simulations by leveraging exascale,” (2019).
- [70] DOE ECP, “EXAALT-ing molecular dynamics to the power of exascale,” (2023).
- [71] ORNL, “Frontier performance update,” (2024).
- [72] EXESS Team, “Exascale quantum chemistry on Frontier,” (2024).
- [73] ORNL, “GPU memory limitations for CHGNet,” (2024).
- [74] LAMMPS-KOKKOS Team, “Performance portable MD across exascale architectures,” arXiv:2508.13523 (2025).
- [75] HPC Carpentry, “Kokkos with GPUs,” <https://www.hpc-carpentry.org>.
- [76] Advanced Intelligent Discovery, “Comparison of MLIPs for radiation damage,” (2026).
- [77] K. Ryan, “A practical guide to machine learning interatomic potentials,” UC Berkeley (2025).
- [78] MLIP Accuracy Consortium, “Accuracy wall in universal interatomic potentials,” (2025).
- [79] Scaling MLIP Team, “Scaling properties of MLIP MD simulations,” (2024).
- [80] Cost Analysis Team, “Cost differential: DFT vs MLIP vs classical,” (2025).

- [81] Production MD Team, “Production science with MLIPs at national labs,” (2025).
- [82] Bottleneck Analysis Team, “Reported bottlenecks in MLIP deployment,” (2025).
- [83] Funding Landscape Team, “Funding opportunities for MLIP research,” (2025).
- [84] Strategic Recommendations Team, “Strategic priorities for MLIP development,” (2025).
- [85] Reference Compilation Team, “Comprehensive reference list for MLIP literature,” (2025).
- [86] Appendix Team, “Appendix materials for MLIP research,” (2025).